5.11.1 Description

In this lab, you'll create a Laravel web app that demonstrates different types of **Dependency Injection (DI)** and **service lifetimes** using Laravel's **Service Container**. You'll inject a service into a controller, bind interfaces to concrete implementations, and explore the difference between **transient (default)** and **singleton** bindings.

> *⚠ Note: Laravel singleton bindings persist **within the same application lifecycle**. When using `php artisan serve`, each request restarts the Laravel app, so singleton instances are **re-created on every refresh**. To observe real singleton behavior, resolve the service **multiple times within the same request** or use Laravel Octane for persistent lifecycles.*

5.11.2 Objectives

By the end of this lab, you will:

- Understand and apply constructor and method injection
- Bind interfaces to implementations using Laravel's service container
- Use singleton and transient (default) service lifetimes
- View DI output in a simple Blade view

5.11.3 Prerequisites

- Laravel 12 installed via Composer
- PHP ≥ 8.2
- Composer and Visual Studio Code
- Basic understanding of Laravel routes and controllers

> *This lab uses **PHP 8.4**, **Laravel 12** and **Visual Studio Code** as the primary editor.*

5.11.4 Steps

Here's a step-by-step guide to creating a simple web app that demonstrates **Dependency Injection** in Laravel 12.